Course Code: CSA1741 Course Name: ARTIFICIAL INTELLIGENCE
Slot: B

ASSIGNMENT 1 – CO1

Nature of Assignment: Individual Duration: 1 Week Total Marks: 50

Student Name: Naresh Kanna. C. M Reg. No: 192412093

CO1: Foundation and history of AI, intelligent agents (agents and environments, rationality, nature of environments, structure of agents), problem-solving agents, problem formulation, searching for solutions, uninformed search strategies and search with partial information.

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Hospital Medicine-Delivery Robot

Scenario — Hospital Medicine-Delivery Robot

A multi-block hospital uses an autonomous robot to carry medicines from the central pharmacy to wards. The building is modelled as a graph of locations (junctions, corridors, lifts, ward doors). The robot may move between connected locations and every move has a travel-time cost.

As new wings are added, the state space of the robot may grow:

1,000 → 10,000 → 100,000 → 1,000,000+ states

You are required to investigate, design and compare uninformed search strategies by which the robot finds a route from the pharmacy to a requested ward as the hospital grows.

Your solution must remain practical for both small and very large state spaces. Any preprocessing (for example, building adjacency lists or a duplicate-detection structure) must be included in your efficiency analysis.

Design Challenge (Day 4): The robot's localisation sensor now fails intermittently, so the robot no longer knows exactly where it is. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Design the Agent and the Search Algorithms (5 Marks)

Before designing algorithms, define the agent and the problem:

- Specify the PEAS description of the robot and classify its environment (observable, deterministic, episodic/sequential, static/dynamic, discrete/continuous, single/multi-agent), justifying each choice
- State which view of AI (acting rationally, thinking rationally, acting humanly, thinking humanly) the robot follows and why it is a rational problem-solving agent. Relate the problem type briefly to the early history of AI problem solving
- Formulate the problem: states, initial state, actions, transition model, goal test, path cost

Then design and propose at least **three meaningfully different** uninformed search strategies for finding a route. Iterative and recursive implementations of the same underlying strategy will not be considered two different approaches.

Your proposed approaches must include:

- At least one iterative approach (queue- or stack-based)
- At least one recursive approach
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required, such as adjacency-list construction or an explored set

You are expected to select the approaches independently. The appropriateness and diversity of the selected strategies are part of the assessment.

Evidence Expected: PEAS and environment classification, complete problem formulation, three distinct strategies with pseudocode/flowcharts, preprocessing requirements and justification for each.

TASK 2 – Identify the Basic Operation (2 Marks)

For each proposed algorithm, identify the basic operation that predominantly determines its execution time.

Discuss:

- What the basic operation is (for example, node expansion, successor generation, goal test or frontier insertion)
- Where it occurs in the algorithm

- Why it is appropriate for efficiency analysis

- How the number of times it executes may change as the state-space size increases

Do not simply name an operation. Relate it to the behaviour of the algorithm.

Evidence Expected: Correct identification and justification of the basic operation for each algorithm.

TASK 3 – Derive the Time- and Space-Complexity Functions Mathematically (6 Marks)

Derive the time and space complexity for each proposed approach using mathematical analysis.

Discuss:

- The input size (branching factor b, shallowest-goal depth d, maximum depth m, optimal cost C^*, minimum step cost ε)
- The number of times the basic operation executes, derived by summing the nodes at each level
- The mathematical expression for time and for memory (frontier size)
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the mathematical reasoning. Merely stating that an algorithm is O(b^d) will not receive full credit.

Evidence Expected: A clear derivation of time and space functions for each approach, including operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4 Marks)

Analyse each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the best case (for example, the goal lies on the first branch explored)
- What condition produces the average case
- What condition produces the worst case (for example, goal at maximum depth, goal unreachable, cheap-edge detours)

- How the number of nodes generated changes in each case, and why

Do not simply provide a table of complexity values. Explain the conditions that lead to each case.

Evidence Expected: A justified best-, average- and worst-case analysis supported by mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis (3 Marks)

Express and compare the growth behaviour of the algorithms using O, Ω and/or Θ.

Discuss:

- How each algorithm behaves as the state space becomes very large
- How the growth rates of the approaches differ in time and in memory
- What the growth means when the hospital grows from 1,000 to 1,000,000+ states, for example, whether memory or time becomes the limiting factor first

Interpret the behaviour in the context of the hospital robot rather than merely stating the notation.

Evidence Expected: Correct asymptotic representation and a meaningful comparison of growth behaviour.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

For the recursive approach, formulate a recurrence relation directly from the behaviour of your algorithm.

Identify clearly:

- The size of the original problem (depth limit or remaining depth)
- The size of the recursive subproblem
- The number of recursive calls (related to the branching factor)
- The work performed outside the recursive calls
- The base case
- How these components lead to the recurrence

The recurrence must be derived from your algorithm, not copied from a textbook.

Evidence Expected: A correct recurrence with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation (4 Marks)

Solve the recurrence from Task 6 using an appropriate analytical method (substitution, iteration, recursion tree or another method).

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the recursive algorithm as depth or branching factor increases
- How the result extends to iterative deepening, where the recursive search is repeated for increasing limits

Show the analytical process, not only the final result.

Evidence Expected: A complete solution with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms (4 Marks)

Implement all proposed algorithms in Python and test them experimentally.

Use generated hospital graphs of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 states

Ensure that:

- All algorithms return correct routes
- The same or equivalent graphs and start/goal pairs are used for comparison
- Testing conditions are consistent and the random seeds are reported
- Test cases include a reachable goal, an unreachable goal and a start equal to the goal
- The implementation corresponds to the algorithms analysed in Tasks 1–7

The purpose of implementation is to generate evidence for validating your theoretical analysis.

Evidence Expected: Working code, test inputs, expected behaviour and actual outputs, with a reproducible testing procedure.

TASK 9 – Record Experimental Performance (3 Marks)

Collect and present performance data for each algorithm across the different state-space sizes.

Record, with code and outputs as proof:

- Nodes generated and nodes expanded
- Maximum frontier size (memory proxy)
- Solution depth and path cost, and whether the path is optimal
- Execution time (averaged over at least 5 runs)

Present the results using tables and graphs (use a log scale where appropriate) so that approaches can be compared easily.

Evidence Expected: Systematically collected experimental data with clear tables and graphs.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

Compare your theoretical analysis with the experimental results.

Discuss:

- Whether the observed growth of nodes generated and of frontier size agrees with the theoretical complexity
- Which algorithm scales better as the state space increases
- Any differences between theory and observation, and the possible reasons (graph structure, duplicates, tie-breaking, language overhead)

Interpret the results rather than simply reproducing tables or graphs.

Evidence Expected: A reasoned comparison between theoretical and observed performance.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

Based on your mathematical analysis and experimental findings, recommend the most appropriate search strategy for the hospital robot.

Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives

- Its theoretical efficiency and experimentally observed performance
- Any preprocessing cost, trade-offs or limitations
- Whether it remains practical as the hospital grows

Your recommendation must be based on your own evidence. Simply choosing the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected: A technically justified recommendation supported by analysis and experiment.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

After a power restart or sensor fault, the robot no longer knows exactly where it is. It knows only that it is at one of *k* possible junctions. It must still reach the pharmacy docking station, whichever junction it is actually at.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

- What has changed from the original problem and which original assumptions (fully observable, known start state) are affected?
- Why does the robot's knowledge now form part of the state?

B. Evaluation of the Original Design – 2 Marks

- Can your original algorithm still be applied? What does a "state" become?
- How large can the new search space be for *k* possible locations, and is the original design acceptable?

C. Adaptation or Redesign – 2 Marks

- Describe the revised strategy (for example, search over belief states, or search with local sensing and a conditional plan), what has changed and the effect on complexity
- If you retain the original solution, give sufficient technical justification

D. Evidence-Based Defence – 2 Marks

- Defend your decision using mathematical analysis and a small experiment (belief states generated and plan length for k = 2, 4, 8, with and without a local sensor), with code and output as proof.

MARK DISTRIBUTION – ASSIGNMENT_1

Task	Marks
Task 1 – Agent and Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3
Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

CSA1741 Assignment 1 – Hospital Medicine-Delivery Robot

Nov 1, 2026 · @Saala

Recommendation: uniform-cost search (UCS) on a precomputed adjacency list, backed by BFS for hop-count queries; IDDFS is rejected because its time explodes on graphs with loops.

Task 1 – Agent and Algorithm Design

Notation. V = number of states (n), E = number of corridors, b = branching factor, d = depth of the shallowest goal (in hops), m = maximum depth, C* = optimal cost, ε = minimum step cost. In my generated hospitals E = 1.5 V (average degree 3, maximum degree 5, edge times 1–10).

1.1 PEAS description

Element	Description
Performance measure	Deliver the medicine to the requested ward; minimise total travel time; no collisions; low battery use; never get lost
Environment	Multi-block hospital: junctions, corridors, lifts, ward doors, staff and patients
Actuators	Drive wheels, lift-button arm, door opener, locked medicine compartment, speaker/buzzer
Sensors	Localisation sensor (RFID/LiDAR/odometry), obstacle sensors, camera, ward-tag reader, battery sensor

1.2 Environment classification (for the planning model)

Property	Choice	Justification
Observable	Fully observable (original design)	The localisation sensor gives the exact node; the map is known. (Day 4 makes it partially observable.)
Deterministic	Deterministic	A move from node v along corridor (v,u) ends at u; slips and delays are ignored at the planning level.
Episodic / sequential	Sequential	Each move changes the robot's position and so changes all later costs and options.
Static / dynamic	Static (while planning)	The map does not change while the route is computed. Crowding is handled by re-planning.
Discrete / continuous	Discrete	Locations form a finite graph; actions are "go to neighbour".
Single / multi-agent	Single-agent	Other people and robots are treated as obstacles,

Property	Choice	Justification
		not as opponents that must be modelled.

1.3 View of AI and rationality

The robot follows **acting rationally**. It is a goal-based problem-solving agent: it selects the action sequence that maximises its performance measure (shortest delivery time) given the percepts and the map it knows. It formulates a goal (reach ward t), formulates a problem, searches for a solution, then executes it. This is the classical approach of early AI problem solving: the Logic Theorist and General Problem Solver (Newell, Simon and Shaw, 1956–57) treated problem solving as search through a space of states. Route finding is the same idea as maze solving (Moore, 1959) and shortest paths (Dijkstra, 1959).

1.4 Problem formulation

- **States:** the robot's location $v \in V$ (junction, corridor point, lift or ward door); $|V| = n$.
- **Initial state:** the pharmacy node s.
- **Actions:** ACTIONS(v) = { go(u) : $(v,u) \in E$ }.
- **Transition model:** RESULT(v, go(u)) = u.
- **Goal test:** v = t, the requested ward.
- **Path cost:** the sum of travel times $c(v,u) \geq \varepsilon > 0$. An optimal solution minimises total time.

1.5 Preprocessing common to all algorithms

Build adjacency lists in CSR form (arrays indptr, nbr, wt) from the edge list in O(V + E). BFS uses a visited/parent array (O(V) to initialise). UCS uses a best-cost table and a closed array. IDDFS uses only a path set of size O(d) to avoid loops on the current path.

1.6 The three strategies and why they were selected

Strategy	Style	Frontier structure	Finds	Why selected
A1: Breadth-First Search (BFS)	Iterative	FIFO queue	Fewest hops	Simplest complete baseline; shows the cost of a wide frontier
A2: Uniform-Cost Search (UCS)	Iterative	Priority queue ordered by g(n)	Least travel time	The real objective is time, not hops, and edges have different costs
A3: Iterative-Deepening DFS (IDDFS)	Recursive (depth-limited search)	Call stack	Fewest hops	Memory-light alternative; tests whether saving memory is worth repeated work

The three differ in data structure (queue, heap, stack), in the optimality criterion (hops vs cost) and in the time/space trade-off. A plain recursive and an iterative DFS would be the same strategy, so only one DFS-family method is used.

1.7 Pseudocode

A1 – BFS (iterative)

```
BFS(s, t):
  if s == t: return [s]
  parent[s] = NIL; queue = [s]
  while queue not empty:
    u = dequeue(queue)                 # node expansion
    for each neighbour v of u:         # successor generation
      if v not in parent:
        parent[v] = u
        if v == t: return path(parent, t)   # goal test at generation
        enqueue(queue, v)
  return FAILURE
```

A2 – UCS (iterative)

```
UCS(s, t):
  best[s] = 0; pq = {(0, s)}
  while pq not empty:
    (g, u) = extract-min(pq)
    if u already closed: continue      # stale entry
    close u
    if u == t: return path(parent, t)  # goal test at expansion => optimal
    for each (v, w) of u:
      if g + w < best[v]:              # relaxation
        best[v] = g + w; parent[v] = u; insert(pq, (g + w, v))
  return FAILURE
```

A3 – IDDFS with recursive depth-limited search

```
IDDFS(s, t):
  for L = 0, 1, 2, ...:
    r = DLS(s, t, L)
    if r is a path: return r
    if r == FAIL: return FAILURE       # whole component searched, goal absent

DLS(u, t, limit):                      # recursive
  if u == t: return [u]
  if limit == 0: return CUTOFF
  cutoff = false; add u to path
  for each neighbour v of u not on path:
    r = DLS(v, t, limit - 1)
    if r is a path: return [u] + r
    if r == CUTOFF: cutoff = true
  remove u from path
  return CUTOFF if cutoff else FAIL
```

Task 2 – Basic Operation

Algorithm	Basic operation	Where it occurs	Why appropriate	How its count changes as the hospital grows
BFS	Successor generation with duplicate check: examine edge (u,v), test "v already seen?", and if not, insert v into the queue	Inner for-loop of the expansion step	Every other step (dequeue, goal test, enqueue) runs at most once per execution of it and costs O(1), so it bounds the running time	Runs once per edge leaving each expanded node, so it equals the sum of degrees of expanded nodes, at most 2E. It grows like b^d until the frontier covers the whole graph, then saturates at 2E = O(V).
UCS	Edge relaxation: compare g(u)+w with best[v] and, if better, push onto the priority queue	Inner for-loop after a node is extracted	Same bound as BFS, but each successful relaxation costs O(log F) for a heap of size F, so the number of heap pushes is the dominant weight	Same count as BFS (at most 2E) plus extra re-pushes whenever a cheaper path is found later. It grows with the number of nodes whose cost is below C*, about b^(C*/ε).
IDDFS	Node generation = one recursive call of DLS (goal test, cycle check and loop set-up)	The first lines of DLS, executed on every call	Each call does O(1) work apart from the successor loop, which is counted in the calls it makes	Counted over all iterations: nodes are regenerated in every deeper iteration. It grows like b^d and is not capped by V, because the same junction is reached again by every different loop-free path.

The count matters because it behaves differently in each algorithm. BFS and UCS stop growing when all V nodes are seen (the duplicate check prunes), so their basic-operation count is bounded by the size of the building. IDDFS has no explored set (it must save memory), so its count tracks the number of loop-free paths, which grows exponentially with depth even if the building is small.

Task 3 – Time and Space Complexity (derivation)

Input size. b = branching factor (successors per node, excluding the parent), d = depth of the shallowest goal, m = maximum depth, C* = optimal cost, ε = minimum step cost. The solution depth for UCS is $\lfloor C^*/\varepsilon \rfloor$. Level i of the search tree contains at most b^i nodes.

3.1 BFS

Goal test is done at generation, so levels 0 to d−1 are expanded and level d is generated.

Expansions (worst case): $1 + b + b^2 + \ldots + b^{(d-1)} = (b^d - 1)/(b - 1)$.

Nodes generated (basic operation count): 1 + b + … + b^d = (b^(d+1) − 1)/(b − 1) = O(b^d).

```
T_{BFS}(d) = \sum_{i=0}^{d} b^i = \frac{b^{d+1}-1}{b-1} = \Theta(b^d)
```

Space: the frontier holds the whole last level plus the explored set, so S_BFS = O(b^d). On a graph with V nodes, both are capped: T = O(min(b^d, V + E)) and S = O(min(b^d, V)).

Preprocessing: adjacency lists O(V + E) and the visited array O(V). With E = 1.5V this is Θ(V).

Overall: T_total = Θ(V + E) + O(min(b^d, V + E)) = Θ(V). For an easy query (small d) the preprocessing dominates, so it must be done once and reused for all deliveries.

3.2 UCS

UCS expands every node with path cost g < C* (and some with g = C*). Since each edge costs at least ε, the solution lies at most ⌊ C*/ε⌋ steps deep, and the goal test at expansion adds one more level:

```
T_{UCS} = O\!\left(b^{\,1+\lfloor C^*/\varepsilon \rfloor}\right) \text{ node expansions, each heap operation } O(\log F)
```

So T = O(b^(1+⌊ C*/ε⌋) · log F), and S = O(b^(1+⌊ C*/ε⌋)) for the heap plus the best-cost table. On a graph, T = O((V + E) log V) and S = O(V). If all edges cost ε then C*/ε = d and UCS behaves like BFS with an extra log factor.

Preprocessing: adjacency lists O(V + E) and the cost table O(V). **Overall:** Θ(V + E) + O((V + E) log V) = O(V log V).

3.3 IDDFS

A depth-limited search with limit L generates N(L) = 1 + b + … + b^L = (b^(L+1) − 1)/(b − 1) nodes. IDDFS repeats it for L = 0, 1, …, d, so the total is a sum of sums. Each level i appears in every iteration with L ≥ i, that is (d − i + 1) times:

```
T_{ID}(d) = \sum_{L=0}^{d} \sum_{i=0}^{L} b^i = \sum_{i=0}^{d} (d-i+1)\, b^i = \frac{b^{d+2} - b - (d+1)(b-1)}{(b-1)^2}
```

Bounding: T_ID ≤ b^d · Σ_j (j+1) b^(−j) ≤ b^d / (1 − 1/b)² = O(b^d). Relative to the last iteration alone, the repeated work is only a factor b/(b−1) (about 1.6 for b = 2.6), because the deepest level dominates.

Space: the call stack holds one path of at most d nodes plus the path set, so S_ID = O(d) (O(bd) if all siblings are stacked). With limit m, O(m).

Preprocessing: adjacency lists O(V + E) only; no explored set. **Overall:** Θ(V) + O(b^d).

Important: because no explored set is kept, the b^d here counts loop-free paths, not distinct nodes. On a graph with loops, T_ID can exceed V by orders of magnitude.

3.4 Summary

Algorithm	Time	Space	Preprocessing	Optimal for	Complete
BFS	O(b^d), at most O(V+E)	O(b^d), at most O(V)	Θ(V+E)	hops	yes
UCS	O(b^(1+⌊ C*/ε⌋) log	O(b^(1+⌊ C*/ε	Θ(V+E)	travel time	yes (ε > 0)

Algorithm	Time	Space	Preprocessing	Optimal for	Complete
	F), at most O((V+E) log V)	⌋)), at most O(V)			
IDDFS	O(b^d) paths, not bounded by V	O(d)	Θ(V+E)	hops	yes

Task 4 – Best, Average and Worst Case

BFS

- **Best case:** start = goal (1 node generated, 0 expanded, tested), or the goal is the first neighbour of the pharmacy (2 generated, 1 expanded). Work is O(1).
- **Average case:** the goal is equally likely to be any node on its level d. All levels 0 to d−1 are expanded, and on average half of level d is generated before the goal appears: ≈ (b^d − 1)/(b − 1) + b^d / 2.
- **Worst case:** the goal is the last node generated on level d, or the goal is unreachable. Then every reachable node is generated, V nodes and 2E edges. Observed: unreachable goal on 1,000 nodes gave 1,000 expanded; the farthest ward at 1,000,000 states gave 1,000,000 generated.

UCS

- **Best case:** the goal is the cheapest neighbour of the start. The start is expanded, then the goal is popped: 2 expansions.
- **Average case:** the goal is at a typical cost C*. All nodes with g < C* are expanded, roughly b^(C*/ε) in a tree and a constant fraction of V in a loopy building.
- **Worst case:** cheap-edge detours. If the direct corridor costs 10 but a longer route of cost-1 corridors is cheaper, UCS first expands everything cheaper than C*. Number of expanded nodes grows with C*/ε, not with d. An unreachable goal expands all V nodes with O(E log V) heap work. Observed: for the farthest ward, UCS found a 22-hop route of cost 75 while BFS's 18-hop route cost 99, at the price of 5× more time.

IDDFS

- **Best case:** the goal's depth is d and it lies on the first branch tried in the last iteration. Earlier iterations must still fail completely, so the cost is Σ_{L<d} N(L) + (d+1) = Θ(b^(d−1)). If start = goal, 1 node. If d = 1 and the goal is the first neighbour: 1 + 2 = 3 nodes.
- **Average case:** the goal is somewhere in the last iteration, so on average half of it is explored: about Σ_{L<d} N(L) + N(d)/2.
- **Worst case:** the goal is the last leaf visited at depth d: T_ID(d) from Task 3. For an unreachable goal the search ends only when every loop-free path from the start has been tried, which is exponential in V on a graph with loops. Observed: a 30-state graph needed 702,669 nodes; a 1,000-state graph exhausted the 3,000,000-node budget (did not finish).

The cases arise from the graph structure, not just the algorithm: BFS cost depends on how many nodes lie before the goal level, UCS on how many are cheaper than the goal, IDDFS on how many loop-free paths of length ≤ d exist.

Task 5 – Asymptotic Analysis

Algorithm	Time	Space	Notes
BFS	Best Ω(1); worst O(b^d) tree, O(V+E) graph; Θ(V) on the farthest ward	Θ(V) worst (frontier ≈ 30% of V)	Linear in the hospital size
UCS	Best Ω(1); worst O((V+E) log V)	Θ(V) worst	Linear plus a log factor from the heap
IDDFS	Best Ω(b^(d−1)); worst O(b^d) loop-free paths, not bounded by V	Θ(d), grows as log V	Smallest memory, largest time

Growth with hospital size. Real hospitals have bounded degree (b ≤ 5), so as V grows the depth d grows only like log V (measured farthest ward: 9, 14, 15, 18 hops for 10^3, 10^4, 10^5, 10^6 states). Hence b^d is of the same order as V for a tree-like building.

- **BFS and UCS** grow Θ(V) in both time and memory. Going from 1,000 to 1,000,000 states multiplies work and frontier by 1,000.
- **IDDFS** has memory Θ(log V), only 9 to 18 stack frames, but its time is b^d over loop-free paths. Every extra corridor loop multiplies the number of paths, so time rises far faster than V.

Which limit is hit first. For BFS/UCS the frontier plus explored set is Θ(V): at a million states the frontier already held 294,456 entries, and at 100 million states it would be about 30 million entries. On a robot controller with small RAM, memory is the first limit. On a server, time (linear in V) is reached later. For IDDFS memory never becomes a problem, but time becomes the first and only limit at about 10,000 states for far wards (it exceeded 3,000,000 node generations).

Task 6 – Recurrence Relation (recursive IDDFS / depth-limited search)

The recursive approach is DLS(u, t, limit) from the pseudocode. Let T(l) be the number of nodes generated (recursive calls made) by one call with remaining depth l.

- **Size of the original problem:** the depth limit L, that is remaining depth l = L.
- **Subproblem size:** each recursive call is made with limit − 1, so l − 1.
- **Number of recursive calls:** one per successor not already on the path, at most b.
- **Work outside the recursive calls:** goal test, cycle check, loop set-up: a constant c per call (taking edge examination as part of the b calls).
- **Base case:** l = 0: the call tests the goal and returns CUTOFF, T(0) = 1.

Combining:

$$T(l) = b \cdot T(l-1) + c, \quad T(0) = 1$$

For the full IDDFS the outer loop repeats DLS for L = 0 to d:

```
T_{ID}(d) = \sum_{L=0}^{d} T(L)
```
This follows directly from the code: the recursive call dls(v, limit − 1, path) is inside the loop over successors (up to b of them), and the first line of dls increments the generated counter by one (the constant work).

Task 7 – Solving the Recurrence

Method: backward substitution (iteration). The recurrence has the "decrease by one, multiply by b" form T(l) = b·T(l−1) + c. The Master theorem needs a divide-and-conquer form T(n) = aT(n/b') + f(n), so it does not apply. Backward substitution works because each step reduces l by exactly 1 and the pattern is visible after a few steps.

Steps (c = 1, T(0) = 1):

```
T(l) = b T(l-1) + 1
     = b ( b T(l-2) + 1 ) + 1           = b^2 T(l-2) + (b + 1)
     = b^2 ( b T(l-3) + 1 ) + (b + 1)   = b^3 T(l-3) + (b^2 + b + 1)
     ...
     = b^k T(l-k) + (b^(k-1) + ... + b + 1)
```
Stop at k = l, where T(0) = 1:

```
T(l) = b^{l}\cdot 1 + \sum_{i=0}^{l-1} b^i = b^l + \frac{b^l-1}{b-1} = \frac{b^{l+1}-1}{b-1} = \Theta(b^l)
```
Check by induction. Base: T(0) = (b − 1)/(b − 1) = 1. Step: b·(b^(l+1) − 1)/(b − 1) + 1 = (b^(l+2) − b + b − 1)/(b − 1) = (b^(l+2) − 1)/(b − 1). ✓

Recursion-tree check. Level i has b^i calls, each doing c work, over levels 0 to l: c · Σ b^i, the same sum.

Result and meaning. T(l) = Θ(b^l): the work is exponential in the depth. Each extra hop in the route multiplies the work by b (about 2.6 on average, up to 4 near the pharmacy in my graphs). Raising b from 2 to 4 at depth 10 changes b^10 from about 1,000 to about 1,000,000.

Extension to iterative deepening. T_ID(d) = Σ_{L=0}^{d} (b^(L+1) − 1)/(b − 1) = (b^(d+2) − b − (d+1)(b − 1))/(b − 1)² = Θ(b^d). Compared with a single search of depth d, T(d) = (b^(d+1) − 1)/(b − 1), the ratio tends to b/(b − 1) (1.63 for b = 2.6). So repeating the search for every limit costs only a constant factor, and the growth stays exponential in d.

Task 8 – Implementation and Testing

Full Python code is in the Appendix (file `hospital_search.py`). It implements exactly the three algorithms of Task 1 (BFS with deque, UCS with heapq, recursive IDDFS) with counters for nodes generated, nodes expanded, edges examined and maximum frontier.

Test setup (reproducible).

- Hospital generator: random spanning tree (degree cap 5) plus 0.5·n extra corridors; edge times are random integers 1–10. **Random seed = 42** for graph and goal selection, so every algorithm sees the identical graph and start/goal pair.
- Sizes: 1,000; 10,000; 100,000; 1,000,000 states. Start (pharmacy) = node 0.
- Two goal scenarios: *typical* (a ward at the middle hop-distance level) and *far* (the farthest ward).

- Each timing is the mean of 5 runs (IDDFS runs that hit the 3,000,000-node budget are run once and marked DNF).
- Environment: Python 3.13, x86-64 cloud container. Run with `python3 hospital_search.py`.

Correctness tests (n = 1,000 plus a sealed 5-node wing). Actual output:

Test case	Algorithm	Result	Verdict
Reachable (node 265)	BFS	5 hops, cost 22, generated 372, expanded 152	PASS, optimal
Reachable	UCS	5 hops, cost 22, generated 756, expanded 380	PASS, optimal
Reachable	IDDFS	5 hops, cost 22, generated 638, expanded 221	PASS, optimal
Start = goal	BFS / UCS / IDDFS	path [0], cost 0, generated 1	PASS (all three)
Unreachable (node 1002)	BFS	no path, 1,000 expanded	PASS
Unreachable	UCS	no path, 1,000 expanded	PASS
Unreachable	IDDFS	node budget (3,000,001) exhausted	PASS (expected exponential worst case)
Unreachable on 30-state graph	IDDFS	no path, terminates, 702,669 generated	PASS
20 random start/goal pairs	all	BFS hops = IDDFS hops, UCS cost ≤ BFS cost, paths valid; mismatches = 0	PASS

Every returned path is checked edge by edge (each step must be an existing corridor) and its cost is recomputed independently. The implementation corresponds line by line with the pseudocode and with the analysed operations: counters increment in the loops identified as basic operations in Task 2.

Task 9 – Experimental Performance

All values come from `results.json` written by the code in the Appendix (seed 42; time = mean of 5 runs). Optimal means the path cost equals UCS's cost. DNF = did not finish within 3,000,000 node generations; its time is for that single capped run. "Max frontier" is queue/heap size for BFS and UCS and recursion depth for IDDFS.

Preprocessing (adjacency-list build): 0.001 s (1k), 0.006 s (10k), 0.07 s (100k), 0.92 s (1M). It is linear in V and is paid once.

9.1 Typical ward (middle hop-distance level)

States n	Algorithm	Generated	Expanded	Max frontier	Hops	Cost	Optimal	Time (ms)
1,000	BFS	372	152	219	5	22	yes	0.13
1,000	UCS	756	380	367	5	22	yes	0.58
1,000	IDDFS	638	221	5	5	22	yes	0.24
10,000	BFS	7,501	4,546	3,017	8	42	no	3.71
10,000	UCS	8,938	5,287	3,582	12	32	yes	9.99
10,000	IDDFS	25,008	9,594	8	8	42	no	10.39
100,000	BFS	25,221	10,455	14,765	8	33	yes	15.34
100,000	UCS	32,097	13,436	18,554	8	33	yes	75.47
100,000	IDDFS	45,106	16,414	8	8	33	yes	37.92
1,000,000	BFS	218,128	90,308	127,820	10	39	yes	184.65
1,000,000	UCS	167,531	66,172	101,114	10	39	yes	469.11
1,000,000	IDDFS	390,803	144,536	10	10	39	yes	351.64

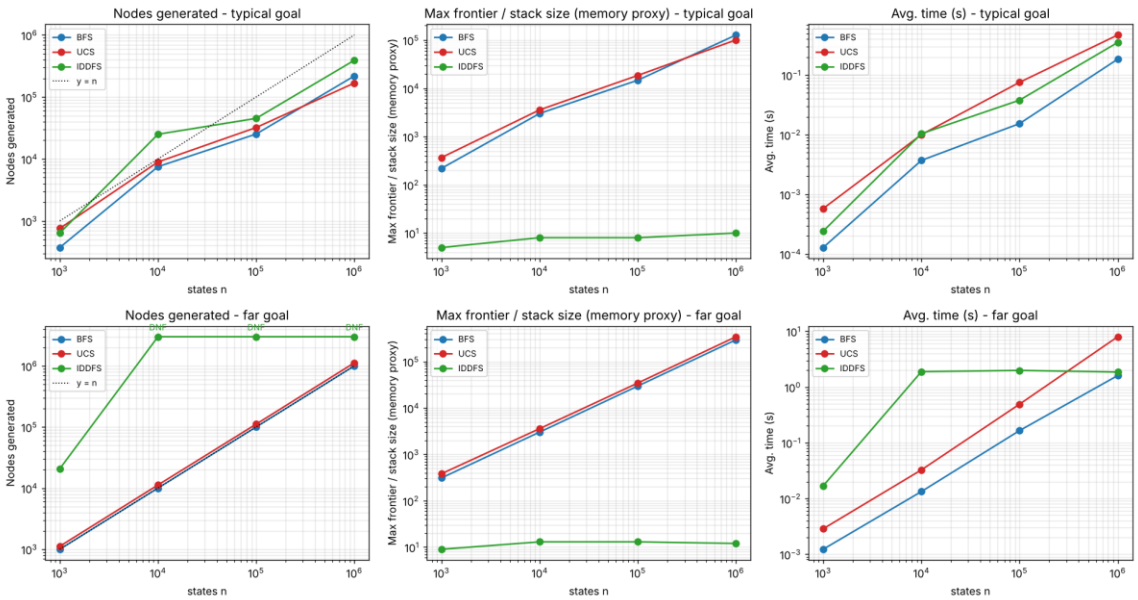
9.2 Farthest ward (stress case)

States n	Algorithm	Generated	Expanded	Max frontier	Hops	Cost	Optimal	Time (ms)
1,000	BFS	997	964	310	9	53	no	1.23
1,000	UCS	1,116	981	382	9	38	yes	2.86
1,000	IDDFS	20,715	8,055	9	9	53	no	16.84
10,000	BFS	10,000	9,998	3,017	14	62	yes	13.30
10,000	UCS	11,233	10,000	3,582	14	62	yes	32.53
10,000	IDDFS	>3,000,000	1,174,181	13	DNF	DNF	n/a	DNF (1,883)
100,000	BFS	99,996	99,948	29,651	15	71	no	165.26
100,000	UCS	111,280	93,139	34,471	15	52	yes	491.95
100,000	IDDFS	>3,000,000	1,177,004	13	DNF	DNF	n/a	DNF (1,984)

States n	Algorithm	Generated	Expanded	Max frontier	Hops	Cost	Optimal	Time (ms)
1,000,000	BFS	1,000,000	999,996	294,456	18	99	no	1,606.13
1,000,000	UCS	1,123,511	998,932	342,697	22	75	yes	7,987.77
1,000,000	IDDFS	>3,000,000	1,167,365	12	DNF	DNF	n/a	DNF (1,855)

9.3 Graphs (log-log scale)

See the figure below (top row typical ward, bottom row farthest ward; left to right: nodes generated, maximum frontier, time). The dotted line on the generated-nodes plots is y = n.



Task 10 – Theory versus Experiment

Nodes generated. For the farthest ward, BFS and UCS generated 997; 10,000; 99,996; 1,000,000 nodes, a log-log slope of 1.00 against n. This matches the theory: the goal level is deeper than the point where b^d exceeds V, so the cost saturates at Θ(V + E). For the farthest ward UCS generated 1.0 to 1.12 times as many nodes as BFS (extra re-insertions), and its time was about 2× to 5× higher because each insertion costs O(log F) and stores a tuple.

Frontier (memory). BFS's maximum frontier was 310; 3,017; 29,651; 294,456 entries, a slope of 0.99, about 30% of V each time. This agrees with the Θ(V) space bound: the widest BFS level of the 1M graph contains 281,649 nodes (levels measured from the pharmacy: 1, 5, 20, 80, 296, …). IDDFS's stack stayed at 9 to 13 frames (slope 0.1 to 0.16), as O(d) predicts, with d growing like log V.

Typical ward. Generated nodes grew more slowly than V (slope 0.76 to 0.88; 218,128 of 1,000,000 for BFS) because the goal lies at depth 10 of an 18-level graph, so the search stops before the frontier

covers the building. Theory predicts b^d here, not V. Time grew at slope ≈ 1.0, a little faster than the node count because larger arrays have worse cache behaviour.

IDDFS versus theory. With b ≈ 3 (between the measured graph-wide mean of 2.6 successors per node and about 4 near the pharmacy), the formula T_ID(d) gives 543 nodes for d = 5 (observed 638). It gives about 14,800 for d = 8 and 132,900 for d = 10 (observed 25,008 to 45,106 and 390,803, within about 3×). For d = 14, 15, 18 it predicts more than 3 million, 10 million, and 100 million nodes, which agrees with every far-ward run exhausting the 3,000,000-node budget. The key difference from BFS is that IDDFS count is not capped at V: 3,000,000 generations on a 10,000-state graph show repeated visits through loops.

Differences and reasons.

- The measured branching factor changes with depth: level sizes grow ×5, ×4, ×4, ×3.7 near the start and then fall below ×1 as the graph saturates, so a single b is an approximation.
- Duplicates: BFS/UCS remove them, IDDFS does not, so theory b^d overstates BFS (capped at V) and understates IDDFS on loopy graphs.
- Tie-breaking and goal position: where the goal sits in the neighbour order changes counts by a few times (the 10,000-state typical run needed 25,008 nodes while the 100,000-state run needed 45,106 for the same depth 8).
- BFS returned a path that was not cost-optimal in 4 of 8 comparable runs, as theory says (it minimises hops); UCS was always optimal.
- Language overhead: Python makes time per node about 1 µs to 8 µs, so only growth rates, not absolute times, should be compared with theory.

Which scales better. BFS and UCS scale linearly (Θ(V)) and finish a 1,000,000-state hospital in 0.2 to 8 seconds. IDDFS does not scale beyond about 10,000 states for distant wards. BFS is the fastest in the hop metric, UCS is the only one that is optimal in travel time.

Task 11 – Final Recommendation

Recommended design: uniform-cost search over a precomputed adjacency list (CSR), with BFS available as a fast hop-count check.

Why UCS. The robot's performance measure is delivery time, and corridors have different travel times. BFS minimises hops, so its route was more costly in 4 of 8 comparable runs (e.g. cost 99 against 75 at 1,000,000 states for the farthest ward). UCS was optimal in all 8 runs.

Why not the alternatives.

- BFS: 2 to 5 times faster than UCS and simpler, but not time-optimal. Keep it only where all corridors take about the same time.
- IDDFS: its memory use is tiny (9 to 13 frames) but it made 3,000,000 generations without finding a far ward at 10,000 states. On a hospital with loops it is not practical, and an unreachable ward cannot be detected in reasonable time.

Evidence.

Measure	UCS result
Theory	O((V+E) log V) time, O(V) space, optimal and complete for $\varepsilon > 0$
1,000,000 states, typical ward	167,531 generated, 0.47 s, optimal
1,000,000 states, farthest ward	1,123,511 generated, 8.0 s, optimal (cost 75 vs BFS 99)
Preprocessing	0.92 s one-off adjacency build, linear in V
Memory	frontier up to 342,697 entries (34% of V)

Trade-offs and limits.

- Memory is $\Theta(V)$, so very large wings on a small controller need more RAM. The adjacency build (about 1 s at 1M states) must be done once, not per delivery.
- The farthest-ward query at 1M states takes about 8 seconds in Python. A compiled implementation, or a one-off Dijkstra from the pharmacy that stores the whole shortest-path tree (O((V+E) log V) once, then O(path length) per ward), removes this. Since the pharmacy is the fixed start, that is the practical deployment.
- UCS is uninformed. An informed heuristic (A*) would reduce expansions, but it is outside this assignment.

Does it remain practical as the hospital grows? Yes: both time and memory are linear in the number of states (up to a log factor), as measured from 1,000 to 1,000,000 states, while IDDFS's time is exponential in depth.

Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt

Decision: MODIFY. Keep the uninformed-search machinery (frontier, explored set, goal test) but search over belief states, and add a local sensor so the plan becomes a conditional plan.

12.1 Impact analysis (2 marks)

- **What changed:** the robot is at one of k possible junctions and must still reach the pharmacy dock, wherever it really is.
- **Assumptions broken:** "fully observable" (the position is no longer sensed) and "known start state" (the start is a set of k states).
- **Why knowledge is part of the state:** the result of an action depends on the true location, which is unknown. What matters for choosing the next action is the set of locations still possible, so that set (the belief state) is what the agent actually holds and what actions transform: RESULT(b, a) = { result(s, a) : $s \in b$ }. A plan is correct only if it works for every member of b.

12.2 Evaluation of the original design (2 marks)

- **Can the original algorithm still be applied?** BFS/UCS can be run unchanged, but on a different graph: a state becomes a belief state $b \subseteq S$, the initial state is the set of k junctions, the goal test is "$b \subseteq \{dock\}$", and a solution is one action sequence that works from every start.

- **Size of the new space:** with N junctions there are 2^N − 1 possible beliefs (about 9×10^15 for N = 53). For deterministic moves a belief never grows, so starting from k states only beliefs of size ≤ k are reachable, at most Σ_{j=1}^{k} C(N, j) = O(N^k):

k	Bound on reachable beliefs (N = 53)	Bound × 4 actions
2	1,431	5,724
4	317,682	1,270,728
8	about 1.07×10^9	about 4.3×10^9

- **Is the original design acceptable?** Only for small k. The original BFS over physical states (53 states) is trivial, but over beliefs it grows as N^k: polynomial for fixed k, yet already too large at k = 8, and exponential (2^N) if k is not limited.

12.3 Adaptation (2 marks)

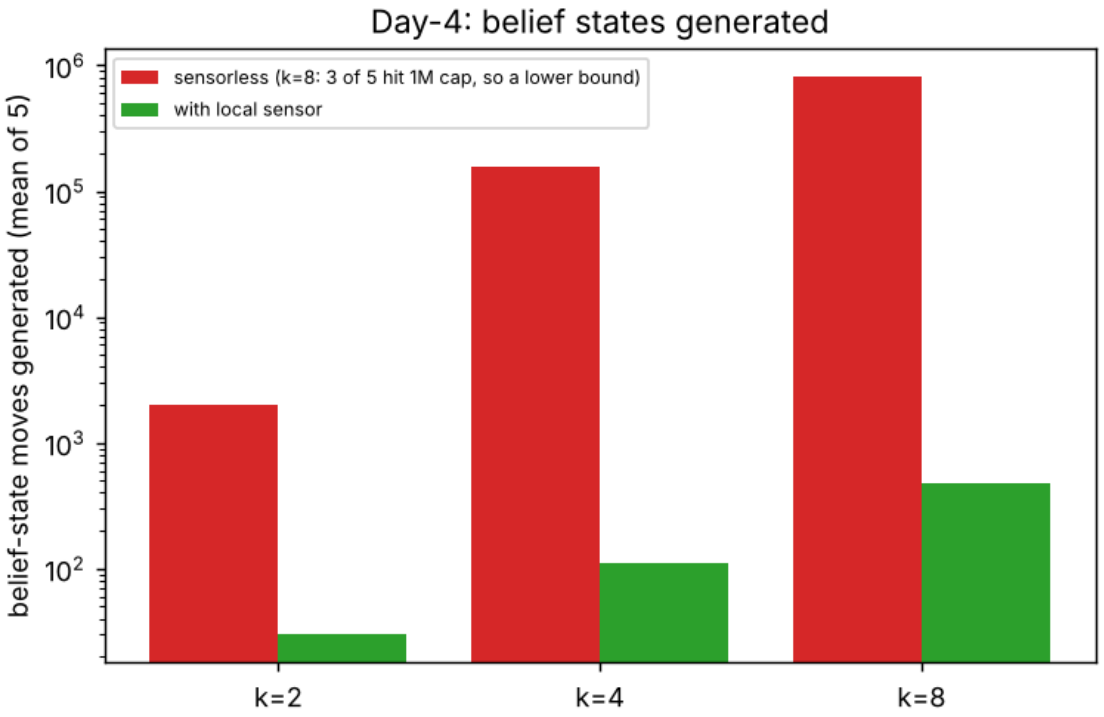
1. **Sensorless version:** BFS over belief states; the answer is one fixed action sequence (a "synchronising" sequence that funnels all k candidates to the dock). Complexity O(N^k · |A|) generated beliefs.
2. **With a local sensor (wall/open-door sensing, N/E/S/W):** after each move the percept splits the belief into smaller groups. The solution becomes a **conditional plan**, found by AND–OR search: an OR choice of action, then an AND over every percept outcome. Singleton beliefs mean the robot knows where it is, so the original single-state search takes over: a one-off Dijkstra/BFS from the dock gives the remaining route cost. The worst-case plan length is found by iterative deepening on the plan depth.
3. **Effect on complexity:** the sensor makes belief sizes drop quickly, so the number of beliefs generated no longer follows N^k but stays small (hundreds), while plan length is no longer than the sensorless plan.

12.4 Evidence-based defence (2 marks)

Experiment (code in Appendix, `belief_search.py`): an 8×8 junction floor with sealed rooms (53 free junctions, dock at (0,0), seed 11); actions N, E, S, W (bumping into a wall leaves the robot where it is); 5 random start sets per k (seeds 100·k + trial). Sensorless search is capped at 1,000,000 generated beliefs.

k	Sensorless: beliefs generated (mean of 5)	Sensorless plan length	With sensor: beliefs generated (mean of 5)	Conditional plan, worst-case length
2	2,007 (5/5 solved)	8.8	30	7.0
4	156,399 (5/5 solved)	12.4	111	11.6
8	more than 811,832 (only 2/5 solved; the other 3 hit the cap)	11.5 (solved cases only)	475	12.0

Solved sensorless cases for k = 8 are the easy ones, so that row is a lower bound for the sensorless cost. Comparing the same start sets, the sensor plan was never longer than the sensorless one (k = 4: 12/12, 14/14, 14/14, 12/12, 6 against 10).



Raw output (proof):

```
floor: 53 junctions, dock=(0, 0), seed=11
 k | sensorless: beliefs generated  plan length | with sensor: beliefs generated
worst-case plan length | trial
 2 |         73      4 |    9      3 | 0
 2 |       3305     12 |   45     12 | 1
 2 |       5161     14 |   40     10 | 2
 2 |        217      5 |   36      5 | 3
 2 |       1277      9 |   20      5 | 4
   mean: sensorless solved 5/5, gen=2007, plan=8.8 | sensor solved 5/5, gen=30,
plan=7.0
 4 |     144073     12 |   88     12 | 0
 4 |     277217     14 |  167     14 | 1
 4 |     214657     14 |   70     14 | 2
 4 |     131397     12 |  107     12 | 3
 4 |      14653     10 |  123      6 | 4
   mean: sensorless solved 5/5, gen=156399, plan=12.4 | sensor solved 5/5, gen=111,
plan=11.6
 8 |  >1000002 (DNF) DNF |  693     12 | 0
 8 |  >1000001 (DNF) DNF |  456     14 | 1
 8 |  >1000001 (DNF) DNF |  340     14 | 2
 8 |     739149     12 |  317     10 | 3
 8 |     320009     11 |  570     10 | 4
   mean: sensorless solved 2/5, gen=811832, plan=11.5 | sensor solved 5/5, gen=475,
```

```
plan=12.0
upper bound on belief space for N=53 junctions: 2^N - 1 = 9.007e+15
```

Defence of the decision. (1) Mathematics: sensorless beliefs grow as N^k, and the measurements agree: 2,007 → 156,399 → more than 811,832 beliefs for k = 2, 4, 8, always below the bound in 12.2 and unaffordable at k = 8. (2) Experiment: the local sensor cut the beliefs generated by 67× (k = 2), 1,400× (k = 4) and more than 1,700× (k = 8) with equal or shorter plans. (3) Retaining the original physical-state BFS unchanged is not possible, since it assumes a known start; redesigning from scratch is unnecessary because the same search machinery and the same final single-state search are reused. Hence **modify**: belief-state search + local sensing + the original UCS once the robot is localised.

Appendix – Python Code

Requires Python 3 and numpy. Save as `hospital_search.py` and `belief_search.py`, run each with `python3 <file>`.

A. hospital_search.py (Tasks 8–10)

```
"""
Hospital medicine-delivery robot: uninformed search (CSA1741 Assignment 1, CO1)
Algorithms: BFS (queue), Uniform-Cost Search (priority queue), Iterative Deepening
DFS (recursive DLS)
Run:  python3 hospital_search.py            -> tests + experiments (writes
results.json)
"""
import random, time, heapq, json, sys
from collections import deque
from array import array
import numpy as np

SEED = 42
MAX_DEG = 5          # corridor/junction degree cap  (so b <= 5)
EXTRA = 0.5          # extra loop corridors = 0.5 * n   (average degree ~ 3)

# ------------------------------------------------------------------ graph generation
def make_edges(n, seed=SEED, isolated=0):
    """Random connected hospital graph on n nodes (+ `isolated` unreachable nodes).
    Nodes 0..n-1 connected; nodes n..n+isolated-1 form a separate island (a sealed
wing)."""
    rng = random.Random(seed)
    deg = [0] * (n + isolated)
    edges = []
    for i in range(1, n):                    # random spanning tree, degree capped
        while True:
            p = rng.randrange(i)
            if deg[p] < MAX_DEG:
                break
        edges.append((p, i, rng.randint(1, 10)))
        deg[p] += 1; deg[i] += 1
    seen = set((min(a, b), max(a, b)) for a, b, _ in edges)
    target = int(EXTRA * n); added = 0
    while added < target:                    # extra corridors (loops)
        a, b = rng.randrange(n), rng.randrange(n)
        if a == b or deg[a] >= MAX_DEG or deg[b] >= MAX_DEG: continue
```

```
        key = (min(a, b), max(a, b))
        if key in seen: continue
        seen.add(key); edges.append((a, b, rng.randint(1, 10)))
        deg[a] += 1; deg[b] += 1; added += 1
    for j in range(n, n + isolated - 1):      # island: a small chain
        edges.append((j, j + 1, rng.randint(1, 10)))
    return n + isolated, edges

class Graph:
    """CSR adjacency lists. Building them is the PREPROCESSING step: O(V+E)."""
    def __init__(self, n, edges):
        t0 = time.perf_counter()
        e = np.array(edges, dtype=np.int64)
        src = np.concatenate([e[:, 0], e[:, 1]]); dst = np.concatenate([e[:, 1], e[:,
0]])
        w = np.concatenate([e[:, 2], e[:, 2]])
        order = np.argsort(src, kind="stable")
        src, dst, w = src[order], dst[order], w[order]
        indptr = np.zeros(n + 1, dtype=np.int64)
        np.add.at(indptr, src + 1, 1); indptr = np.cumsum(indptr)
        self.n = n
        self.indptr = array('i', indptr.astype(np.int32).tobytes())
        self.nbr = array('i', dst.astype(np.int32).tobytes())
        self.wt = array('i', w.astype(np.int32).tobytes())
        self.build_time = time.perf_counter() - t0
        self.m = len(edges)

    def succ(self, u):
        ip = self.indptr
        return range(ip[u], ip[u + 1])

def path_cost(g, path):
    tot = 0
    for a, b in zip(path, path[1:]):
        ws = [g.wt[k] for k in g.succ(a) if g.nbr[k] == b]
        if not ws: return None            # invalid move
        tot += min(ws)
    return tot

# ---------------------------------------------------------------- result container
class Res:
    def __init__(self):
        self.path = None; self.cost = None; self.generated = 0; self.expanded = 0
        self.max_frontier = 0; self.edges_examined = 0; self.dnf = False

# ---------------------------------------------------------------- 1. BFS (iterative,
FIFO queue)
def bfs(g, s, t):
    r = Res()
    if s == t:
        r.path = [s]; r.cost = 0; r.generated = 1; return r
    parent = array('i', [-2]) * g.n         # explored set + parent pointers  O(V)
init
    parent[s] = -1
    q = deque([s]); r.generated = 1; r.max_frontier = 1
```

```
    nbr, ip = g.nbr, g.indptr
    while q:
        u = q.popleft(); r.expanded += 1
        for k in range(ip[u], ip[u + 1]):
            r.edges_examined += 1
            v = nbr[k]
            if parent[v] == -2:
                parent[v] = u; r.generated += 1
                if v == t:                                  # goal test at generation
                    r.path = _trace(parent, t); r.cost = path_cost(g, r.path); return
r
                q.append(v)
        if len(q) > r.max_frontier: r.max_frontier = len(q)
    return r

def _trace(parent, t):
    p = []
    while t != -1:
        p.append(t); t = parent[t]
    return p[::-1]

# ---------------------------------------------------------------- 2. UCS (iterative,
priority queue)
def ucs(g, s, t):
    r = Res()
    INF = 1 << 60
    best = {s: 0}; parent = {s: -1}; done = bytearray(g.n)
    pq = [(0, s)]; r.generated = 1; r.max_frontier = 1
    nbr, wt, ip = g.nbr, g.wt, g.indptr
    while pq:
        c, u = heapq.heappop(pq)
        if done[u]: continue                            # stale (lazy deletion) entry
        done[u] = 1; r.expanded += 1
        if u == t:                                      # goal test at expansion ->
optimal
            p = []; x = u
            while x != -1: p.append(x); x = parent[x]
            r.path = p[::-1]; r.cost = c; return r
        for k in range(ip[u], ip[u + 1]):
            r.edges_examined += 1
            v = nbr[k]; nc = c + wt[k]
            if nc < best.get(v, INF):
                best[v] = nc; parent[v] = u
                heapq.heappush(pq, (nc, v)); r.generated += 1
        if len(pq) > r.max_frontier: r.max_frontier = len(pq)
    return r

# ---------------------------------------------------------------- 3. IDDFS
(recursive depth-limited search)
class Budget(Exception): pass

def iddfs(g, s, t, budget=3_000_000, max_depth=10**9):
    r = Res()
    nbr, ip = g.nbr, g.indptr
    on_path = set()
```

```
    state = {"maxdepth": 0}

    def dls(u, limit, path):
        """recursive depth-limited search: returns 'found' | 'cutoff' | 'fail'"""
        r.generated += 1
        if r.generated > budget: raise Budget()
        if u == t: return "found"
        if limit == 0: return "cutoff"
        r.expanded += 1
        cutoff = False
        on_path.add(u); path.append(u)
        if len(path) > r.max_frontier: r.max_frontier = len(path)
        for k in range(ip[u], ip[u + 1]):
            r.edges_examined += 1
            v = nbr[k]
            if v in on_path: continue                  # avoid cycles on the CURRENT
path only
            res = dls(v, limit - 1, path)
            if res == "found": return "found"
            if res == "cutoff": cutoff = True
        on_path.discard(u); path.pop()
        return "cutoff" if cutoff else "fail"

    sys.setrecursionlimit(10000)
    try:
        for L in range(0, max_depth):
            path = []; on_path.clear()
            res = dls(s, L, path)
            if res == "found":
                r.path = path + [t]; r.cost = path_cost(g, r.path); r.depth_limit =
L; return r
            if res == "fail": return r                   # whole component exhausted
-> unreachable
    except Budget:
        r.dnf = True
    return r

ALGOS = {"BFS": bfs, "UCS": ucs, "IDDFS": iddfs}

# ------------------------------------------------------------------ helpers
def bfs_levels(g, s):
    dist = array('i', [-1]) * g.n; dist[s] = 0; q = deque([s])
    while q:
        u = q.popleft()
        for k in g.succ(u):
            v = g.nbr[k]
            if dist[v] < 0: dist[v] = dist[u] + 1; q.append(v)
    return dist

def pick_goal(g, s, seed, frac=0.5):
    """choose a ward at a 'typical' BFS depth (median hop distance from the
pharmacy)"""
    dist = bfs_levels(g, s)
    ds = sorted(set(d for d in dist if d > 0))
    D = ds[int(len(ds) * frac)] if ds else 0
```

```
    cand = [v for v in range(g.n) if dist[v] == D]
    return random.Random(seed).choice(cand), D, max(ds)

def valid(g, p, s, t):
    return p is not None and p[0] == s and p[-1] == t and path_cost(g, p) is not None

# ---------------------------------------------------------------- tests (Task 8)
def run_tests():
    print("=== CORRECTNESS TESTS (n=1000, seed=%d, + 5-node sealed wing) ===" % SEED)
    n, e = make_edges(1000, SEED, isolated=5)
    g = Graph(n, e)
    s = 0
    goal, D, _ = pick_goal(g, s, SEED)
    unreachable = 1002          # sealed wing
    cases = [("reachable", s, goal), ("start == goal", s, s), ("unreachable", s,
unreachable)]
    ucs_cost = ucs(g, s, goal).cost
    for name, a, b in cases:
        for an, f in ALGOS.items():
            r = f(g, a, b)
            if name == "unreachable":
                if an == "IDDFS":      # exponential worst case: expected to exhaust
the node budget
                    ok = r.path is None and r.dnf
                    print(f"{name:14s} {an:6s} path=None generated={r.generated:7d}
budget exhausted (DNF)  -> {'PASS (expected worst case)' if ok else 'FAIL'}")
                    continue
                ok = r.path is None and not r.dnf
                print(f"{name:14s} {an:6s} path=None expanded={r.expanded:6d}
generated={r.generated:7d}  -> {'PASS' if ok else 'FAIL'}")
            else:
                ok = valid(g, r.path, a, b)
                extra = ""
                if name == "reachable":
                    extra = f" optimal_cost={r.cost == ucs_cost}"
                print(f"{name:14s} {an:6s} hops={len(r.path)-1:3d} cost={r.cost:4d}
gen={r.generated:6d} exp={r.expanded:6d}{extra} -> {'PASS' if ok else 'FAIL'}")
    # IDDFS termination on a tiny graph with an unreachable goal (n=30)
    n2, e2 = make_edges(30, SEED, isolated=3); g2 = Graph(n2, e2)
    r2 = iddfs(g2, 0, 31, budget=50_000_000)
    print(f"unreachable (n=30) IDDFS path={r2.path} dnf={r2.dnf}
generated={r2.generated} -> {'PASS' if r2.path is None and not r2.dnf else 'FAIL'}")
    # cross-check on 20 random pairs: BFS hops == IDDFS hops ; UCS cost <= BFS cost
    rng = random.Random(7); bad = 0
    for _ in range(20):
        a, b = rng.randrange(1000), rng.randrange(1000)
        rb, ru, ri = bfs(g, a, b), ucs(g, a, b), iddfs(g, a, b, budget=2_000_000)
        if ri.dnf: continue
        if not (len(rb.path) == len(ri.path) and ru.cost <= rb.cost and valid(g,
ru.path, a, b)): bad += 1
    print("random-pair cross-check (20 pairs): mismatches =", bad)

# ---------------------------------------------------------- experiments (Tasks
9-10)
def experiment(sizes=(1000, 10000, 100000, 1000000), runs=5, budget=3_000_000):
```

```
    rows = []
    for scen, frac in (('typical', 0.5), ('far', 0.999999)):
        for n0 in sizes:
            n, e = make_edges(n0, SEED)
            g = Graph(n, e)
            s = 0
            goal, D, ecc = pick_goal(g, s, SEED, frac)
            print(f"\n--- [{scen}] n={n0}  edges={g.m}  avg_deg={2*g.m/n:.2f}
start={s} goal={goal} BFS-depth={D} (eccentricity {ecc})  adjacency
build={g.build_time:.2f}s", flush=True)
            for an, f in ALGOS.items():
                times = []; r = None
                for i in range(runs):
                    t1 = time.perf_counter()
                    r = f(g, s, goal) if an != "IDDFS" else f(g, s, goal, budget)
                    times.append(time.perf_counter() - t1)
                    if r.dnf: break
                row = dict(scenario=scen, n=n0, edges=g.m, algo=an,
build=g.build_time, depth_bfs=D,
                           generated=r.generated, expanded=r.expanded,
edges_examined=r.edges_examined,
                           max_frontier=r.max_frontier, dnf=r.dnf,
                           hops=None if r.path is None else len(r.path) - 1,
cost=r.cost,
                           time=sum(times) / len(times), runs=len(times))
                rows.append(row)
                print(row, flush=True)
            ref = [x for x in rows if x["n"] == n0 and x["scenario"] == scen and
x["algo"] == "UCS"][0]["cost"]
            for x in rows:
                if x["n"] == n0 and x["scenario"] == scen: x["optimal"] = (x["cost"]
== ref) if x["cost"] is not None else None
    json.dump(rows, open("results.json", "w"), indent=1)
    return rows

if __name__ == "__main__":
    run_tests()
    experiment()
```

B. belief_search.py (Task 12)

```
"""
Day-4 challenge: robot knows only that it is in one of k junctions.
Hospital floor = 8x8 junction grid with sealed rooms (walls). Actions: N,E,S,W (bump
= stay).
(a) Sensorless planning: BFS over BELIEF STATES (frozensets of possible junctions) ->
one fixed action sequence.
(b) With a local wall sensor (percept = which of N,E,S,W are open): AND-OR /
conditional plan,
    found by iterative deepening on the worst-case plan length.
Run: python3 belief_search.py
"""
import random
from collections import deque
```

```
W = H = 8
SEED = 11
DIRS = {"N": (-1, 0), "E": (0, 1), "S": (1, 0), "W": (0, -1)}

def build(seed=SEED, wall_frac=0.18):
    rng = random.Random(seed)
    free = {(r, c) for r in range(H) for c in range(W) if rng.random() > wall_frac}
    # keep largest connected component
    seen, best = set(), set()
    for s in free:
        if s in seen: continue
        comp, q = {s}, deque([s]); seen.add(s)
        while q:
            r, c = q.popleft()
            for dr, dc in DIRS.values():
                v = (r + dr, c + dc)
                if v in free and v not in seen: seen.add(v); comp.add(v); q.append(v)
        if len(comp) > len(best): best = comp
    return best

CELLS = build()

def move(s, a):
    r, c = s; dr, dc = DIRS[a]; v = (r + dr, c + dc)
    return v if v in CELLS else s            # bumping into a wall -> stay

def percept(s):
    return tuple(a for a in "NESW" if (s[0] + DIRS[a][0], s[1] + DIRS[a][1]) in
CELLS)

DOCK = min(CELLS)                              # pharmacy docking station (top-left-
most free cell)

# physical-state distances to the dock (used for singleton beliefs, i.e. once
localised)
def dist_to_dock():
    d = {DOCK: 0}; q = deque([DOCK])
    while q:
        u = q.popleft()
        for a in "NESW":
            v = move(u, a)
            if v not in d: d[v] = d[u] + 1; q.append(v)
    return d
DIST = dist_to_dock()

def is_goal(b): return all(s == DOCK for s in b)

# ---------- (a) sensorless: BFS over belief states
def sensorless(b0, limit=1_000_000):
    start = frozenset(b0)
    if is_goal(start): return [], 1
    seen = {start}; q = deque([(start, [])]); gen = 1
    while q:
        b, plan = q.popleft()
        for a in "NESW":
```

```
            nb = frozenset(move(s, a) for s in b); gen += 1
            if nb in seen: continue
            seen.add(nb)
            if is_goal(nb): return plan + [a], gen
            q.append((nb, plan + [a]))
            if gen > limit: return None, gen
    return None, gen                          # no sensorless plan exists

# ---------- (b) with local sensor: conditional plan; worst-case length by iterative
deepening
def with_sensor(b0):
    start = frozenset(b0); gen = [0]; memo = {}
    def solve(b, depth):
        """True if a conditional plan of worst-case length <= depth exists from
belief b"""
        if is_goal(b): return True
        if len(b) == 1:                           # localised -> physical shortest
path
            return DIST[next(iter(b))] <= depth
        if depth == 0: return False
        key = (b, depth)
        if key in memo: return memo[key]
        ok = False
        for a in "NESW":
            nb = frozenset(move(s, a) for s in b); gen[0] += 1
            if nb == b: continue
            groups = {}
            for s in nb: groups.setdefault(percept(s), set()).add(s)   # the sensor
splits the belief
            if all(solve(frozenset(g), depth - 1) for g in groups.values()):
                ok = True; break
        memo[key] = ok
        return ok
    for L in range(0, 200):
        if solve(start, L): return L, gen[0]
    return None, gen[0]

if __name__ == "__main__":
    print(f"floor: {len(CELLS)} junctions, dock={DOCK}, seed={SEED}")
    print(f"{'k':>2} | {'sensorless: belief states generated':>36} {'plan
length':>11} | {'with sensor: belief states generated':>38} {'worst-case plan
length':>22} | trial")
    for k in (2, 4, 8):
        res = []
        for trial in range(5):
            rng = random.Random(100 * k + trial)
            b0 = rng.sample(sorted(CELLS), k)
            plan, g1 = sensorless(b0)
            L, g2 = with_sensor(b0)
            res.append((g1, None if plan is None else len(plan), g2, L))
            print(f"{k:>2} | {(str(g1) if plan is not None else '>'+str(g1)+'
(DNF)'):>36} {str('DNF' if plan is None else len(plan)):>11} | {g2:>38} {str(L):>22}
| {trial}")
        ok = [r for r in res if r[1] is not None]
        print(f"   mean over 5 trials: sensorless solved {len(ok)}/5, gen={sum(r[0]
```

```
for r in res)/5:.0f}, plan={(sum(r[1] for r in ok)/len(ok)) if ok else
float('nan'):.1f} | sensor solved 5/5, gen={sum(r[2] for r in res)/5:.0f},
plan={sum(r[3] for r in res)/5:.1f}")
    print("upper bound on belief space for N=%d junctions: 2^N - 1 = %.3e" %
(len(CELLS), 2.0**len(CELLS) - 1))
```